Actividad 4.2

Curso: Desarrollo de aplicaciones avanzadas de ciencias computacionales (Gpo 570)
Profesor: Luis Ricardo Peña Llamas
Alumno: Fernando Daniel Monroy Sánchez (A01750536)
Fecha: 15 de octubre de 2026

---

1. Introducción

Una melodía se define como una secuencia temporal de notas discretas con alturas y duraciones determinadas. En teoría de la información y sistemas estocásticos, surge la pregunta de si dicha secuencia puede modelarse formalmente como un proceso estocástico.

Un proceso estocástico es una colección de variables aleatorias $\{X_t : t \in T\}$ indexadas en el tiempo. En la música tonal occidental, la sucesión de notas no es ruido caótico ni tampoco una trayectoria estrictamente determinista: responde a principios de consonancia física, tensión armónica y resolución. Por ello, la música constituye un proceso estocástico con estructura de dependencia temporal.

El objetivo principal de esta práctica consiste en:

1. Diseñar una cadena de Markov de tiempo discreto sobre la escala de Re Mayor (D Major) y acordes armónicos.
2. Representar la dinámica mediante una matriz de transiciones y validar sus propiedades estocásticas.
3. Implementar un generador pseudoaleatorio de transiciones melódicas y acústicas.
4. Sintetizar físicamente cada tono con frecuencia y duración variable usando envolvente ADSR para evitar distorsión.
5. Analizar la convergencia hacia la distribución estacionaria mediante las ecuaciones de Chapman Kolmogorov.
6. Elaborar una reflexión sobre patrones repetitivos, aprendizaje de estilos y limitaciones del modelo.

(Ver Apéndice A para el código de configuración y parámetros)

Resumen del Sistema y Síntesis Acústica

Parámetro	Valor
Tonalidad base	Re Mayor (D Major)
Estados melódicos	7 notas (D4 a C#5)
Estados armónicos	4 acordes (D, G, A, Bm)
Frecuencia muestreo	44,100 Hz (Calidad CD)
Afinación	12 TET (A4 = 440 Hz)
Articulación	Senoidal con ADSR
Duraciones	4 figuras (120 BPM)

2. Desarrollo

2.1. Escala diatónica

Conforme a los principios de física acústica analizados por Lemnismath, el sonido musical se fundamenta en ondas periódicas caracterizadas por amplitud y frecuencia. Al pulsar una cuerda o columna de aire no se emite una frecuencia aislada sino armónicos enteros $f_k = k \cdot f_1$.

La consonancia depende de la coincidencia espectral: la octava (razón 2:1) aporta identidad tonal y la quinta justa (razón 3:2) aporta equilibrio acústico. Al encadenar 12 quintas sucesivas surge una discrepancia respecto a 7 octavas conocida como coma pitagórica (1.01364). El sistema de temperamento igual de 12 tonos (12 TET) resuelve esto dividiendo la octava en 12 semitonos idénticos con razón constante:

$$r = 2^{1/12} \approx 1.059463094$$

La frecuencia f de cada nota se obtiene desde su índice MIDI n referenciado a La 4 ($A_4 = 440$ Hz, MIDI 69):

$$f(n) = 440 \cdot 2^{\frac{n-69}{12}} \text{ Hz}$$

(Ver Apéndice B para el código de cálculo acústico)

Grados de la Escala Diatónica de Re Mayor

Nota	Grado	MIDI	Frecuencia (Hz)	Razón a Tónica
D4	I (Tónica)	62	293.66	1.0000
E4	ii (Supertónica)	64	329.63	1.1225
F#4	iii (Mediante)	66	369.99	1.2599
G4	IV (Subdominante)	67	392.00	1.3348
A4	V (Dominante)	69	440.00	1.4983
B4	vi (Submediante)	71	493.88	1.6818
C#5	vii° (Sensible)	73	554.37	1.8877

Nota sobre temperamento acústico: La razón constante $2^{1/12}$ permite transportar melodías preservando intervalos, asegurando consonancia uniforme en todo el registro.

2.2. Matriz de transición

Una cadena de Markov homogénea y de tiempo discreto se formaliza como una tupla $(S, P, \pi^{(0)})$:

- $S = \{s_1, \dots, s_m\}$: Espacio discreto de estados (notas o acordes).
- $P \in \mathbb{R}^{m \times m}$: Matriz de probabilidades de transición con $P_{ij} = P(X_{t+1} = s_j \mid X_t = s_i)$.
- $\pi^{(0)} \in \mathbb{R}^m$: Vector de distribución inicial.

Propiedad de Markov:
El estado futuro depende únicamente del estado presente:

$$P(X_{t+1} = s_j \mid X_t = s_i, \dots, X_0 = s_0) = P(X_{t+1} = s_j \mid X_t = s_i) = P_{ij}$$

Esta memoria nula se asemeja al juego de Serpientes y Escaleras visto en clase, donde la casilla futura depende solo de la casilla actual y el dado, a diferencia de extraer cartas de una baraja sin reemplazo donde la baraja cambia con cada extracción previa.

Como expuso el profesor Ricardo Peña en clase con el ejemplo de los registros de 20 días con 11 días de estudio, el conteo frecuentista simple (11 sobre 20) ignora el orden temporal. Mientras un registro responde a una regla estocástica (70% de estudiar si se estudió hoy), el otro proviene de un volado independiente (50 sobre 50). El análisis de bigramas permite estimar transiciones y modelar la dependencia serial. Siguiendo el planteamiento

de la actividad, por ejemplo, la probabilidad de cambiar a la nota SI (B4) dado que actualmente suena la nota RE (D4) se fija en $P(B4 \mid D4) = 0.10$.

Las probabilidades se definieron según la armonía funcional:

- Tónica (D4): Centro de gravedad tonal de máxima estabilidad.
- Supertónica (E4): Grado de paso que conduce a la dominante (A4, 25%) o tónica (D4, 30%).
- Mediante (F#4): Nota modal que conecta con la subdominante (G4, 30%) y tónica (D4, 20%).
- Subdominante (G4): Tensión intermedia orientada a la dominante (A4, 35%).
- Dominante (A4): Máxima tensión armónica que resuelve en tónica (D4, 50%) o submediante (B4, 15%).
- Submediante (B4): Centro relativo menor que conduce a subdominante (G4, 25%) o dominante (A4, 20%).
- Sensible (C#5): Inestabilidad melódica con resolución ascendente obligada a tónica (D4, 70%).

(Ver Apéndice C para el código de matriz y verificación)

Matriz de Probabilidades de Transición P

Estado (t)	D4	E4	F#4	G4	A4	B4	C#5
D4	0.10	0.20	0.15	0.20	0.20	0.10	0.05
E4	0.30	0.05	0.20	0.10	0.25	0.05	0.05
F#4	0.20	0.20	0.05	0.30	0.15	0.05	0.05
G4	0.15	0.10	0.25	0.05	0.35	0.05	0.05
A4	0.50	0.10	0.10	0.05	0.05	0.15	0.05
B4	0.20	0.15	0.15	0.25	0.20	0.05	0.00
C#5	0.70	0.10	0.05	0.00	0.10	0.05	0.00

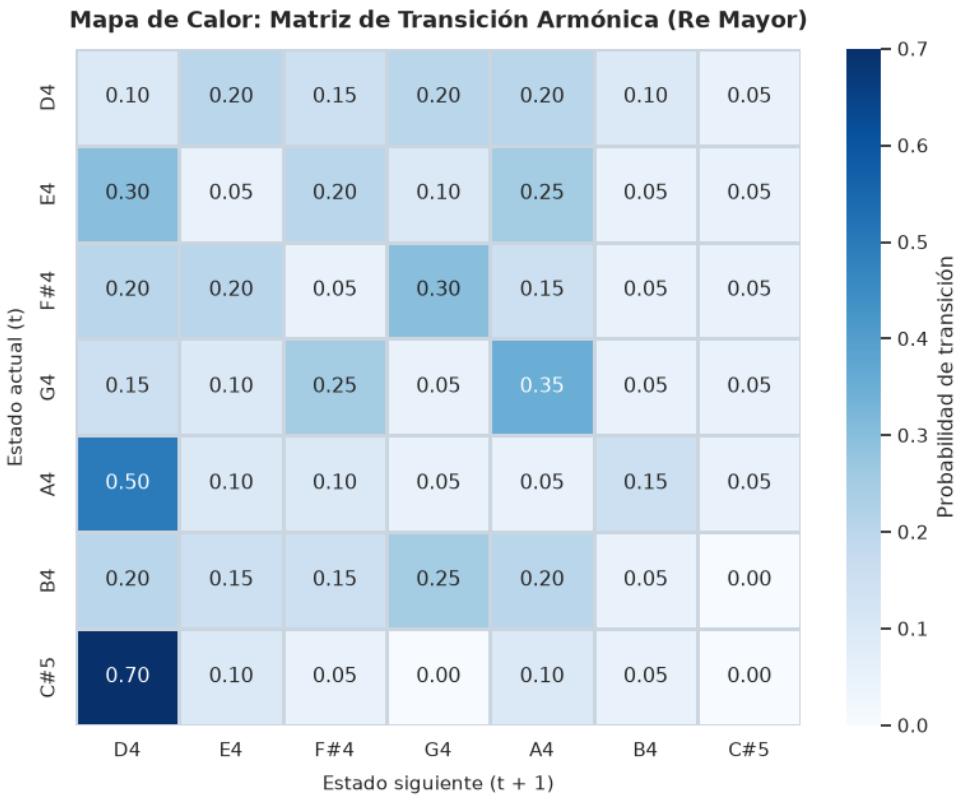
```
¿Probabilidades >= 0? : True
Suma por fila de la matriz:
[1. 1. 1. 1. 1. 1. 1.]
¿Filas suman 1.0 (allclose)? : True
```

Nota sobre la matriz estocástica: Cada fila representa la distribución condicional $P(X_{t+1} \mid X_t = s_i)$, sumando estrictamente 1.0.

2.3. Mapa de calor

La siguiente figura ilustra visualmente la densidad de transición. Las celdas más oscuras indican las resoluciones armónicas predominantes (sensible C#5 a tónica D4 con 0.70, y dominante A4 a tónica D4 con 0.50):

(Ver Apéndice D para el código de graficación)



Nota sobre resoluciones armónicas: La tónica actúa como sumidero probabilístico natural del sistema, atrayendo al quinto grado dominante y al séptimo grado sensible.

2.4. Distribución estacionaria

Según las ecuaciones de Chapman Kolmogorov, la distribución tras n transiciones se obtiene mediante potencias matriciales:

$$\pi^{(n)} = \pi^{(0)} \cdot P^n$$

En el modelo de lenguaje de tres palabras presentado por Rodrigo Castellanos en clase (”él”, ”gato”, ”duerme”), al partir de 100 textos iniciales distribuidos como [50, 40, 10], el vector evoluciona tras 2 pasos a [0.29, 0.33, 0.36] y se estabiliza cerca de 20 transiciones. Esta distribución límite π^* satisface:

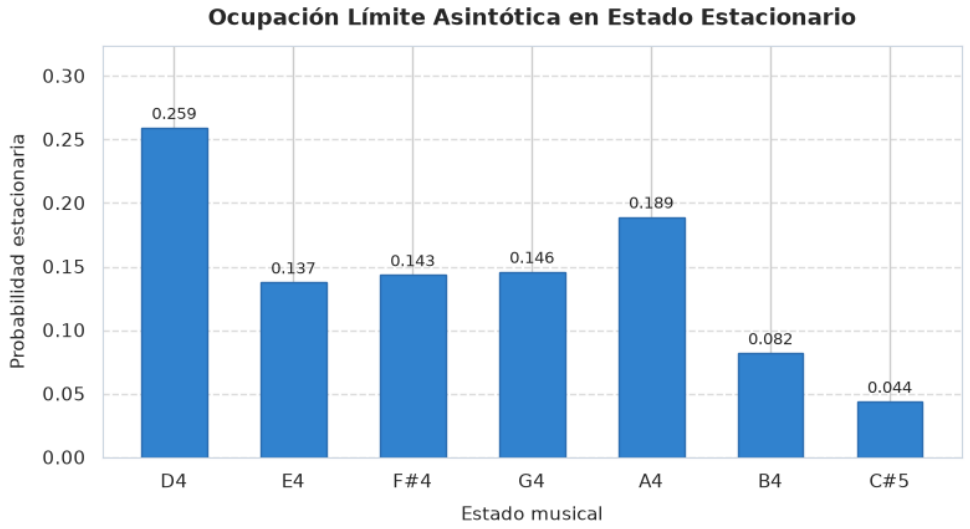
$$\pi^* \cdot P = \pi^* \quad \text{con} \quad \sum_{i=1}^{m} \pi_i = 1$$

Este principio sustenta el algoritmo PageRank de Larry Page en Google, modelando la navegación web como una caminata aleatoria.

(Ver Apéndice D para el cálculo de estado estacionario)

Distribución Estacionaria Asintótica

Nota	Grado	Probabilidad límite	Porcentaje
D4	I (Tónica)	0.2591	25.91%
E4	ii (Supertónica)	0.1375	13.75%
F#4	iii (Mediante)	0.1433	14.33%
G4	IV (Subdominante)	0.1457	14.57%
A4	V (Dominante)	0.1889	18.89%
B4	vi (Submediante)	0.0818	8.18%
C#5	vii° (Sensible)	0.0437	4.37%



Nota sobre convergencia asintótica: La tónica D4 (25.91%) y la dominante A4 (18.89%) acumulan cerca del 45% del tiempo en estado estacionario, confirmando el reposo consonante del sistema.

2.5. Generador de melodías

Para determinar cada transición se emplea el método de transformada inversa sobre la distribución acumulada del estado actual.

Las funciones de síntesis reciben frecuencia fundamental (f) y duración (d). Para evitar discontinuidades de fase que generen chasquidos acústicos, se aplicó una envolvente ADSR:

- Attack (0.04 s): Crecimiento lineal de 0 a 1.
- Decay (0.04 s): Descenso al nivel de sostenimiento (0.85).
- Sustain: Mantenimiento de amplitud.
- Release (0.05 s): Extinción a cero antes del siguiente tono.

Siguiendo la recomendación de clase de variar duraciones para mayor naturalidad, se muestrean 4 figuras a 120 BPM: negra (0.50 s, 50%), corchea (0.25 s, 25%), negra con puntillo (0.75 s, 15%) y blanca (1.00 s, 10%).

(Ver Apéndice E para el código del generador y exportación WAV)

```
Paso 0 (Inicio): Nota D4 | Frecuencia: 293.66 Hz | Duración: 1.00 s
Paso 01: D4 a F#4 | 369.99 Hz | 0.75 s
Paso 02: F#4 a G4 | 392.00 Hz | 0.50 s
Paso 03: G4 a B4 | 493.88 Hz | 0.75 s
Paso 04: B4 a F#4 | 369.99 Hz | 1.00 s
Paso 05: F#4 a E4 | 329.63 Hz | 0.75 s
Paso 06: E4 a A4 | 440.00 Hz | 0.50 s
Paso 07: A4 a D4 | 293.66 Hz | 0.75 s
```

```
Paso 08: D4 a G4 | 392.00 Hz | 0.50 s
Paso 09: G4 a F#4 | 369.99 Hz | 0.50 s
Paso 10: F#4 a E4 | 329.63 Hz | 0.50 s
Paso 11: E4 a F#4 | 369.99 Hz | 0.25 s
Paso 12: F#4 a E4 | 329.63 Hz | 0.75 s
Paso 13: E4 a A4 | 440.00 Hz | 0.50 s
Paso 14: A4 a D4 | 293.66 Hz | 1.00 s
Paso 15: D4 a A4 | 440.00 Hz | 0.25 s
Paso 16: A4 a E4 | 329.63 Hz | 1.00 s
```

A continuación se presenta el registro detallado de transiciones paso a paso:

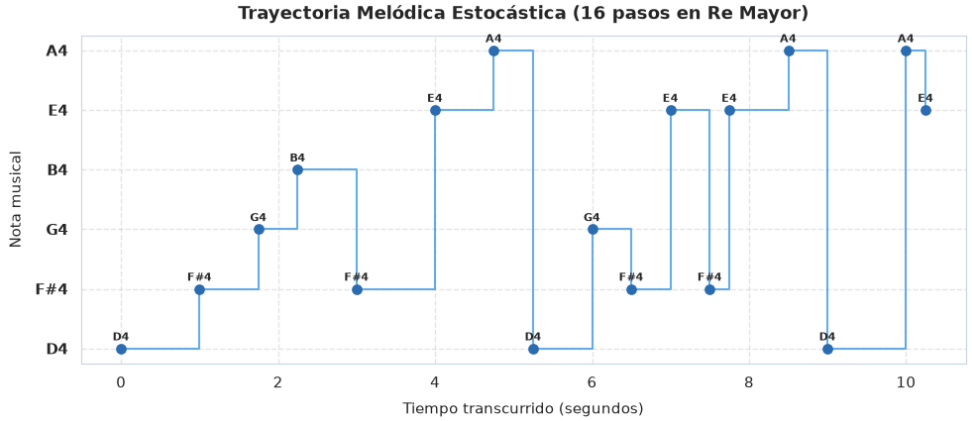
Secuencia: 16 Transiciones

Paso	Estado ant.	Nota	Frecuencia	Duración	Tiempo acum.
0	Inicio	D4	293.66	1.0	0.0
1	D4	F#4	369.99	0.75	1.0
2	F#4	G4	392.0	0.5	1.75
3	G4	B4	493.88	0.75	2.25
4	B4	F#4	369.99	1.0	3.0
5	F#4	E4	329.63	0.75	4.0
6	E4	A4	440.0	0.5	4.75
7	A4	D4	293.66	0.75	5.25
8	D4	G4	392.0	0.5	6.0
9	G4	F#4	369.99	0.5	6.5
10	F#4	E4	329.63	0.5	7.0
11	E4	F#4	369.99	0.25	7.5
12	F#4	E4	329.63	0.75	7.75
13	E4	A4	440.0	0.5	8.5
14	A4	D4	293.66	1.0	9.0
15	D4	A4	440.0	0.25	10.0
16	A4	E4	329.63	1.0	10.25

Reproducción interactiva de la melodía generada de 16 transiciones:

```
<IPython.lib.display.Audio object>
```

Trayectoria melódica en el tiempo (Piano Roll):



Nota sobre síntesis acústica: La envolvente ADSR suprime chasquidos en los tonos senoidales y genera transiciones limpias.

2.6. Armonización

Para evaluar texturas a mayor escala, se generó una secuencia de 32 pasos y una pieza polifónica a 8 compases donde una cadena gobierna la progresión de acordes ($D - G - A - Bm$) mientras otra genera la línea melódica simultáneamente:

(Ver Apéndice F para el código de armonización)

Reproductor de la pieza polifónica armonizada (8 compases completos):

```
<IPython.lib.display.Audio object>
```

Nota sobre acompañamiento armónico: El acoplamiento de dos cadenas produce polifonía sin elevar el espacio de estados individual.

3. Reflexión

Uno empieza pensando que la música va a sonar robótica o caótica como en el prototipo de Netlify, donde cada nota tiene casi la misma probabilidad de salir. Pero en cuanto incorporas teoría armónica, la melodía cobra sentido. Como vimos con el profe Ricardo Peña en la clase del 8 de octubre, un proceso markoviano modela la dependencia condicional paso a paso. El caso de los registros de estudio de 20 días con 11 días de estudio lo ilustra bien: el conteo frecuentista no distingue entre seguir una regla o tirar una moneda, pero el análisis de bigramas revela la verdadera estructura. En música ocurre igual, pues la coherencia melódica surge de la probabilidad condicional entre notas consecutivas.

La música real depende de la repetición deliberada de motivos y estribillos, pero una cadena de primer orden padece de amnesia temporal porque solo recuerda el estado actual. Para capturar patrones repetitivos se requeriría ampliar la memoria a órdenes superiores o usar modelos de orden variable, similar al ejercicio en clase del modelo de tres palabras de Rodrigo Castellanos ("él", "gato", "duerme") que estabilizó probabilidades tras 20 pasos, o como hace PageRank con caminantes aleatorios. Sin embargo, aumentar la memoria provoca una explosión en el espacio de estados, haciendo que el modelo memorice el corpus en lugar de generar variaciones fluidas.

El modelo puede aprender la estadística local de un estilo estimando probabilidades por máxima verosimilitud sobre bigramas de un corpus MIDI con suavizado de Laplace. Con esto aprende cadencias comunes y evita quintas paralelas. No obstante, carece por completo de teleología: no tiene dirección formal a largo plazo y no sabe hacia dónde va la pieza tras medio segundo. Además, según la teoría de Lerdahl y Jackendoff, la música se organiza en árboles jerárquicos anidados que un autómata finito regular no puede representar, mientras que incorporar polifonía a cuatro voces dispararía el sistema a más de 20 mil estados.

Al igual que con el análisis de colas M/M/1 y el dimensionamiento de cajas en Walmart discutido en clase, estos modelos sirven para simular escenarios rápidos sin arrastrar historiales pesados. Con duraciones variables y envolvente ADSR para evitar chasquidos en los tonos senoidales, la cadena produce secuencias musicales interesantes con mínimo costo computacional. Aunque la generación actual utilice redes con atención, la cadena de Markov sigue siendo una base intuitiva y elegante para entender procesos estocásticos.

4. Apéndices

Códigos fuente completos para garantizar reproducibilidad total.

Apéndice A: Configuración inicial

```
from pathlib import Path
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from great_tables import GT
from IPython.display import Audio, display

def _gt_to_latex(self):
    latex_str = self.as_latex()
    latex_str = re.sub(
        r"\\begin\{table\}(\[[^\]]*\])?",
        r"\\begin{table}[H]\n\\centering",
        latex_str
    )
    latex_str = latex_str.replace(
        r"\begin{tabular*}{\linewidth}",
        r"\begin{tabular*}{0.75\linewidth}"
    )
    latex_str = re.sub(
        r"\\fontsize\{[^}]+\}\{[^}]+\}\\selectfont",
        r"\\normalsize",
        latex_str
    )
    latex_str = re.sub(
        r"\\caption\*\{\s*\{\\large\s+([^\}]+)\}\s*\}",
        r"\\caption*{\\textbf{\\normalsize \1}}",
        latex_str
    )
    return latex_str

GT._repr_latex_ = _gt_to_latex

sns.set_theme(style="whitegrid", font_scale=0.95)
plt.rcParams["axes.edgecolor"] = "#CBD5E0"
plt.rcParams["axes.linewidth"] = 0.8

overview_data = pd.DataFrame([
    {"Parámetro": "Tonalidad base", "Valor": "Re Mayor (D Major)"},
    {"Parámetro": "Estados melódicos", "Valor": "7 notas (D4 a C#5)"},
    {"Parámetro": "Estados armónicos", "Valor": "4 acordes (D, G, A, Bm)"},
    {"Parámetro": "Frecuencia muestreo", "Valor": "44,100 Hz (Calidad CD)"},
    {"Parámetro": "Afinación", "Valor": "12 TET (A4 = 440 Hz)"},
    {"Parámetro": "Articulación", "Valor": "Senoidal con ADSR"},
    {"Parámetro": "Duraciones", "Valor": "4 figuras (120 BPM)"},
])

gt_overview = (
    GT(overview_data)
    .tab_header(title="Resumen del Sistema y Síntesis Acústica")
    .cols_align(align="left", columns=["Parámetro"])
    .cols_align(align="right", columns=["Valor"])
)
gt_overview
```

Apéndice B: Escala diatónica

```
from act_2.scales import D_MAJOR_NOTES_OCT4, note_to_freq, note_to_midi
import pandas as pd
from great_tables import GT

notes_data = []
for n in D_MAJOR_NOTES_OCT4:
    midi_val = note_to_midi(n)
    freq_val = note_to_freq(n)
    ratio_to_tonic = freq_val / note_to_freq("D4")
    notes_data.append({
        "Nota": n,
        "Grado": {
            "D4": "I (Tónica)",
            "E4": "ii (Supertónica)",
            "F#4": "iii (Mediante)",
            "G4": "IV (Subdominante)",
            "A4": "V (Dominante)",
            "B4": "vi (Submediante)",
            "C#5": "vii° (Sensible)"
        }[n],
        "MIDI": midi_val,
        "Frecuencia (Hz)": f"{freq_val:.2f}",
        "Razón a Tónica": f"{ratio_to_tonic:.4f}"
    })

gt_notes = (
    GT(pd.DataFrame(notes_data))
    .tab_header(title="Grados de la Escala Diatónica de Re Mayor")
    .cols_align(align="left", columns=["Nota", "Grado"])
    .cols_align(
        align="right",
        columns=["MIDI", "Frecuencia (Hz)", "Razón a Tónica"]
    )
)
gt_notes
```

Apéndice C: Matriz de transición

```
import numpy as np
from great_tables import GT
from act_2.scales import (
    D_MAJOR_NOTES_OCT4,
    HARMONIC_TRANSITIONS_D_MAJOR,
)
from act_2.markov import MarkovChain

chain_notes = MarkovChain(
    D_MAJOR_NOTES_OCT4,
    HARMONIC_TRANSITIONS_D_MAJOR
)
df_matrix = chain_notes.to_dataframe()

gt_matrix = (
    GT(df_matrix.reset_index().rename(columns={"index": "Estado (t)"}))
    .tab_header(title="Matriz de Probabilidades de Transición P")
    .fmt_number(columns=D_MAJOR_NOTES_OCT4, decimals=2)
    .cols_align(align="left", columns=["Estado (t)"])
    .cols_align(align="right", columns=D_MAJOR_NOTES_OCT4)
)

row_sums = HARMONIC_TRANSITIONS_D_MAJOR.sum(axis=1)
is_non_negative = bool(np.all(HARMONIC_TRANSITIONS_D_MAJOR >= 0.0))
is_stochastic = bool(np.allclose(row_sums, 1.0, atol=1e-5))

print(f"¿Probabilidades >= 0? : {is_non_negative}")
print(f"Suma por fila de la matriz:\n{row_sums}")
print(f"¿Filas suman 1.0 (allclose)? : {is_stochastic}")
```

Apéndice D: Distribución estacionaria

```
import pandas as pd
import matplotlib.pyplot as plt
from great_tables import GT
from act_2.scales import D_MAJOR_NOTES_OCT4
from act_2.visualizer import (
    plot_transition_heatmap,
    plot_stationary_distribution,
)

fig_heatmap = plot_transition_heatmap(
    chain_notes,
    title="Mapa de Calor: Matriz de Transición Armónica (Re Mayor)"
)
plt.show()

pi_stationary = chain_notes.compute_stationary_distribution()
df_stationary = pd.DataFrame([
    {
        "Nota": note,
        "Grado": {
            "D4": "I (Tónica)",
            "E4": "ii (Supertónica)",
            "F#4": "iii (Mediante)",
            "G4": "IV (Subdominante)",
            "A4": "V (Dominante)",
            "B4": "vi (Submediante)",
            "C#5": "vii° (Sensible)"
        }[note],
        "Probabilidad límite": f"{prob:.4f}",
        "Porcentaje": f"{prob * 100:.2f}%"
    }
    for note, prob in zip(D_MAJOR_NOTES_OCT4, pi_stationary)
])

gt_stationary = (
    GT(df_stationary)
    .tab_header(title="Distribución Estacionaria Asintótica")
    .cols_align(align="left", columns=["Nota", "Grado"])
    .cols_align(
        align="right",
        columns=["Probabilidad límite", "Porcentaje"]
    )
)

fig_stat = plot_stationary_distribution(
    chain_notes,
    title="Ocupación Límite Asintótica en Estado Estacionario"
)
plt.show()
```

Apéndice E: Generador de melodías

```
from pathlib import Path
import matplotlib.pyplot as plt
from great_tables import GT
from IPython.display import Audio, display
from act_2.melody_generator import MelodyGenerator
from act_2.tones import DEFAULT_SAMPLE_RATE, export_wav
from act_2.visualizer import plot_melody_roll

export_dir = Path("export")
if not export_dir.is_dir() and Path("../export").is_dir():
    export_dir = Path("../export")
export_dir.mkdir(parents=True, exist_ok=True)

generator = MelodyGenerator(chain_notes)
N_TRANSICIONES = 16
audio_wave, events_list, df_melody = generator.generate_melody(
    initial_note="D4",
    n_transitions=N_TRANSICIONES,
    seed=101,
    verbose=True
)

output_wav_path = export_dir / "melodia_markov_16.wav"
export_wav(output_wav_path, audio_wave)

df_melody_display = df_melody.rename(columns={
    "Estado anterior": "Estado ant.",
    "Nota / Acorde": "Nota",
    "Frecuencia (Hz)": "Frecuencia",
    "Duración (s)": "Duración",
    "Tiempo acumulado (s)": "Tiempo acum."
})

gt_melody = (
    GT(df_melody_display)
    .tab_header(title=f"Secuencia: {N_TRANSICIONES} Transiciones")
    .cols_align(align="center", columns=["Paso"])
    .cols_align(align="left", columns=["Estado ant.", "Nota"])
    .cols_align(
        align="right",
        columns=["Frecuencia", "Duración", "Tiempo acum."]
    )
)

display(Audio(audio_wave, rate=DEFAULT_SAMPLE_RATE))

fig_roll = plot_melody_roll(
    sequence=[e.curr_state for e in events_list],
    durations=[e.duration_s for e in events_list],
    title="Trayectoria Melódica Estocástica (16 pasos en Re Mayor)"
)
plt.show()
```

Apéndice F: Armonización

```
from pathlib import Path
from IPython.display import Audio, display
from act_2.scales import CHORD_STATES, CHORD_TRANSITION_MATRIX
from act_2.markov import MarkovChain
from act_2.tones import DEFAULT_SAMPLE_RATE, export_wav

export_dir = Path("export")
export_dir.mkdir(parents=True, exist_ok=True)

audio_32, events_32, df_32 = generator.generate_melody(
    initial_note="D4",
    n_transitions=32,
    seed=2024,
    verbose=False
)
export_wav(export_dir / "melodia_markov_32.wav", audio_32)

chain_chords = MarkovChain(CHORD_STATES, CHORD_TRANSITION_MATRIX)
mix_audio, df_harm = generator.generate_harmonized_track(
    melody_chain=chain_notes,
    chord_chain=chain_chords,
    initial_note="D4",
    initial_chord="D",
    bars=8,
    beats_per_bar=4,
    tempo_bpm=120.0,
    seed=777
)
export_wav(export_dir / "melodia_armonizada_8compases.wav", mix_audio)
display(Audio(mix_audio, rate=DEFAULT_SAMPLE_RATE))
```